Homework 3

siyul2 Siyu Li

October 2026

Code: https://github.com/SiyuLi915/CS521
Colab's link in one Folder https://drive.google.com/drive/folders/16sHFFm2RsbndwNTQr-dOcyXc0lgOmchp?usp=drive_link

Problem 1

Colab notebook Link: https://colab.research.google.com/drive/1K-A1D8ji7OsyZPOsL7HPRNAXD2uz5xHs?usp=drive_link
I trained both models from scratch for five epochs. The adversarially trained model uses ℓ_∞ PGD with $\epsilon = 8/255$. All accuracies below are percentages.

Model	Clean	1/255	2/255	4/255	8/255	16/255
Standard	58.5	38.4	22.9	6.8	0.5	0.0
Adv. trained	36.9	35.2	32.8	29.7	25.0	12.7

Accuracy under 50-step ℓ_∞ PGD.

Model	Clean	1/255	2/255	4/255	8/255	16/255
Standard	58.5	40.4	27.1	13.1	3.5	0.8
Adv. trained	36.9	35.2	32.8	29.6	24.9	16.0

Accuracy under FGSM.

(a)

Adversarial training improves robustness at larger perturbations, but lowers clean accuracy. The standard model has 58.5% clean accuracy, which drops to 0.5% under PGD at 8/255. The adversarially trained model has 36.9% clean accuracy and retains 25.0% at 8/255. Both models become less accurate as the perturbation grows.

(b)

At 8/255, the adversarially trained model achieves 25.0% under PGD and 24.9% under FGSM, compared with 0.5% and 3.5% for the standard model.

PGD uses multiple steps, while FGSM takes one. PGD is stronger on the standard model at every tested radius. For the adversarially trained model, the attacks are within 0.1 percentage points up to 8/255. At 16/255, PGD is stronger: accuracy is 12.7%, compared with 16.0% under FGSM.

Problem 2: SynCode

Colab notebook Link: https://colab.research.google.com/drive/1pyepEpKWRQZ-v3ezzacALSMBA0yyDaEk?usp=drive_link

```
!pip install -q --ignore-requires-python syncode==0.4.16
import operator
from syncode import Syncode
GRAMMAR = r"""
start: " " number " " OPERATOR " " number "\n"
number: INT "." INT
INT: /[0-9]+/
OPERATOR: "+" | "-" | "*" | "/"
"""
FEW_SHOT = """Extract the two operands and the operator (+, -, *, /) from the query.

Query: What is 15.5 plus 4.25?
Extraction: 15.5 + 4.25

Query: What is 100. minus 37.8?
Extraction: 100.0 - 37.8

Query: What is 6.0 multiplied by 7.?
Extraction: 6.0 * 7.0

Query: What is 81.9 divided by 9.0?
Extraction: 81.9 / 9.0

"""
syn_llm = Syncode(model="microsoft/phi-2", mode="grammar_strict", grammar=GRAMMAR,
    parse_output_only=True, max_new_tokens=30)
OPS = {"+": operator.add, "-": operator.sub, "*": operator.mul, "/": operator.truediv}
def run_syncode(textual_query):
    output = syn_llm.infer(FEW_SHOT + f"Query: {textual_query}\nExtraction:")[0]
    a, op, b = output.split()
    operands = [float(a), float(b)]
    return operands, op, OPS[op](*operands)
for query in ["What is 327. multiplied by 11.0?", "What is 45.1 plus 23.54?", "What is 120.4
    divided by 4.0?"]:
    print(query, run_syncode(query))
```

Output:

```
What is 327. multiplied by 11.0? ([327.0, 11.0], '*', 3597.0)
What is 45.1 plus 23.54? ([45.1, 23.54], '+', 68.64)
What is 120.4 divided by 4.0? ([120.4, 4.0], '/', 30.1)
```

Problem 3

Colab notebook Link: `https://colab.research.google.com/drive/1U9cKpnawM-IjuF_C_cW0lLMVwp5_W0Xh?usp=drive_link`

(a) Single-norm adversarial training

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union
Base	78.7	48.4	59.8	22.4	22.3
ℓ_∞ fine-tuned	78.5	44.7	56.9	20.5	20.4
ℓ_2 fine-tuned	82.6	32.2	60.1	28.3	24.6
ℓ_1 fine-tuned	84.5	0.0	0.1	24.1	0.0

(i) The weakest norm is ℓ_1 for the base, ℓ_∞ fine-tuned, and ℓ_2 fine-tuned models. It is ℓ_∞ for the ℓ_1 fine-tuned model.

(ii) Robustness does not transfer consistently. Fine-tuning on ℓ_2 raises ℓ_1 accuracy from 22.4% to 28.3%, but lowers ℓ_∞ accuracy from 48.4% to 32.2%. Fine-tuning on ℓ_1 nearly eliminates robustness to the other two norms, giving 0% union accuracy.

(b) Multi-norm adversarial training (RAMP)

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Time/epoch (s)
Base	78.7	48.4	59.8	22.4	22.3	–
ℓ_∞ fine-tuned	78.5	44.7	56.9	20.5	20.4	47.2
ℓ_2 fine-tuned	82.6	32.2	60.1	28.3	24.6	47.3
ℓ_1 fine-tuned	84.5	0.0	0.1	24.1	0.0	46.9
RAMP	79.1	41.2	61.9	45.0	39.4	100.5

(i) RAMP has the highest union accuracy (39.4%), compared with 24.6% for the best single-norm model. The ℓ_1 model has the highest clean accuracy (84.5%), but 0% union accuracy.

(ii) RAMP takes about 2.1× as long per epoch (100.5 versus about 47 seconds). It runs both attacks on every image and computes an extra clean gradient for gradient projection.

(c) My method: balanced screening

(i) Idea. The base model is weakest under ℓ_1. I keep both norms equally represented while using only one full attack per image:

1. Run two-step ℓ_1 and ℓ_∞ attacks on each image.
2. Subtract the ℓ_∞ loss from the ℓ_1 loss and rank the images by this difference.
3. Give the top half fresh 10-step ℓ_1 attacks and the rest fresh 10-step ℓ_∞ attacks.
4. Train on the combined adversarial batch using average cross-entropy loss.

The equal split gives both norms enough training examples. The ranking assigns ℓ_1 to images where it is relatively more harmful. This aims to improve union accuracy without running two full attacks per image.

(ii) Implementation. I implement this in `my_loss` and train with the default `adv_train` settings. Implementation notebook.

(iii) Results. Results are averaged over seeds 0 and 1. Time per epoch is the total training time divided by ten.

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Time/epoch (s)
Base	78.7	48.4	59.8	22.4	22.3	–
RAMP	79.0	41.6	62.2	45.8	39.8	100.5
Mine	76.8	43.5	62.0	46.8	**41.4**	68.1

Union accuracies are 39.4% and 40.3% for RAMP, and 41.5% and 41.3% for my method. Since my method is faster, an additional equal-time comparison is not required.

(iv) Ablation. Both ablations use seed 0. *No balancing* chooses the higher-loss norm from screening. *Random norm* skips screening and independently assigns each image either norm with probability 1/2.

Variant	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Time/epoch (s)
Balanced screening	77.3	43.7	62.3	46.9	41.5	68.1
No balancing	77.0	45.2	60.3	42.0	39.7	68.7
Random norm	77.9	42.3	61.3	49.1	41.1	49.0

(v) Analysis.

- My method improves mean union accuracy over RAMP by about 1.6 percentage points with 32% less training time, but lower clean accuracy.
- Balancing raises union accuracy from 39.7% to 41.5%. Without it, only 24% of images receive ℓ_1 attacks initially, falling to about 15% by the end.
- Screening adds only 0.4 points over random assignment while costing 39% more time. This single-seed result suggests that keeping both norms represented matters more than screening.

(vi) Check the evaluation. I test my best run (seed 0) with stronger attacks.

Evaluation	Clean	ℓ_∞	ℓ_2	ℓ_1	Union
Default (50 steps, 2 restarts)	77.3	43.7	62.3	46.9	41.5
Strong (200 steps, 5 restarts)	77.3	43.4	62.3	45.5	41.0

Union accuracy drops by only 0.5 percentage points. This supports the result under stronger attacks, but does not rule out gradient masking. My method adds no test-time randomness or transformation requiring a separate adaptive attack.